C++20 Coroutines - Stackless Cooperative Multitasking

Theory, Edge Cases & Code Examples

- ✓ The Problem They Solve
- ✓ The Three Keywords
- ✓ How They Work Under the Hood
- ✓ Customizing Coroutine Behavior
- ✓ Suspension Points

→ Swipe to learn →

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 1. What Are Coroutines - The Problem They Solve (1/2)

Traditional Async Programming Problems:

```
// ❌ Problem 1: Callback Hell
void download_file(const std::string& url,
                   std::function<void(Data)> on_complete,
                   std::function<void(Error)> on_error) {
    http_get(url, [on_complete, on_error](Result result) {
        if (result.success) {
            parse_data(result.data, [on_complete, on_error](ParseResult pr) {
                if (pr.valid) {
                    save_to_disk(pr.data, [on_complete, on_error](bool ok) {
                        if (ok) {
                            on_complete(pr.data);
                        } else {
                            on_error(Error::DiskError);
                        }
                    });
                } else {
                    on_error(Error::ParseError);
                }
            });
        } else {
            on_error(Error::NetworkError);
        }
    });
}
// Deeply nested, hard to read, difficult error handling
```

```
// ❌ Problem 2: State Machine Complexity
class AsyncOperation {
    enum State { IDLE, CONNECTING, DOWNLOADING, PARSING, SAVING, DONE };
    State state_ = IDLE;
    Data data_;

public:
    void step() {
        switch (state_) {
            case IDLE:
                start_connection();
// ...continues on next slide
```

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 1. What Are Coroutines - The Problem They Solve (2/2)

[Code continues across slides]

```
// ...continued from previous slide
                state_ = CONNECTING;
                break;
            case CONNECTING:
                if (connection_ready()) {
                    start_download();
                    state_ = DOWNLOADING;
                }
                break;
            // ... many more states
        }
    }
};
// Manual state management, error-prone
```

The C++20 Coroutine Solution:

```
// ✅ C++20: Linear, readable async code
Task<Data> download_file(const std::string& url) {
    auto result = co_await http_get(url);          // Suspends here
    auto parsed = co_await parse_data(result);     // Suspends here
    auto saved = co_await save_to_disk(parsed);    // Suspends here
    co_return parsed;                              // Returns result
}
// Looks like synchronous code, but non-blocking!
```

Benefits:

- **Linear flow**: Code reads top-to-bottom
- **Automatic state management**: Compiler handles state transitions
- **Easy error handling**: Use try/catch naturally
- **No callback hell**: Flat structure
- **Composable**: Coroutines can call other coroutines

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 2. Coroutine Fundamentals - The Three Keywords

C++20 introduces three new keywords that make a function a coroutine:

The Three Coroutine Keywords:

Keyword	Purpose	Effect
`co_await`	Suspend and wait for result	Pauses coroutine until awaitable completes
`co_yield`	Produce value and suspend	Returns value to caller, can resume later
`co_return`	Return final value and finish	Terminates coroutine, returns final result

What Makes a Function a Coroutine:

```
// Regular function
int regular_function() {
    return 42;
}

// Coroutine (has co_return)
Task<int> coroutine1() {
    co_return 42;
}

// Generator coroutine (has co_yield)
Generator<int> coroutine2() {
    co_yield 1;
    co_yield 2;
    co_yield 3;
}

// Async coroutine (has co_await)
Task<int> coroutine3() {
    auto result = co_await async_operation();
    co_return result;
}
```

Key Insight: If a function body contains **any** of `co_await`, `co_yield`, or `co_return`, it's a coroutine.

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 3. Coroutine Mechanics - How They Work Under the Hood (1/2)

Coroutine Lifecycle:

```
┌─────────────────────────────────────────────────────────┐
│ 1. INVOCATION                                           │
│    - Caller calls coroutine function                    │
│    - Compiler allocates coroutine frame (on heap usually)│
│    - Coroutine frame stores: locals, parameters, state  │
└─────────────────────────────────────────────────────────┘
                          │
                          │
                          ▼
┌─────────────────────────────────────────────────────────┐
│ 2. INITIAL SUSPENSION                                   │
│    - Promise object created (promise_type)              │
│    - initial_suspend() called                           │
│    - Can suspend immediately or start running           │
└─────────────────────────────────────────────────────────┘
                          │
                          │
                          ▼
┌─────────────────────────────────────────────────────────┐
│ 3. EXECUTION & SUSPENSION POINTS                        │
│    - Coroutine runs until co_await / co_yield / co_return│
│    - At suspension: saves state, returns control to caller│
│    - Can be resumed later via coroutine_handle          │
└─────────────────────────────────────────────────────────┘
                          │
                          │
                          ▼
┌─────────────────────────────────────────────────────────┐
│ 4. FINAL SUSPENSION                                     │
│    - final_suspend() called                             │
│    - Can keep frame alive or destroy it                 │
└─────────────────────────────────────────────────────────┘
                          │
                          │
                          ▼
┌─────────────────────────────────────────────────────────┐
│ 5. DESTRUCTION                                          │
│    - Coroutine frame deallocated                        │
│    - Local variables destroyed                          │
│    - Promise object destroyed                           │
└─────────────────────────────────────────────────────────┘
```

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 3. Coroutine Mechanics - How They Work Under the Hood (2/2)

Coroutine Frame:

When you call a coroutine, the compiler creates a **coroutine frame** (similar to stack frame, but heap-allocated):

```
struct CoroutineFrame {
    void* resume_point;         // Where to resume (instruction pointer)
    void* destroy_point;        // Cleanup routine
    promise_type promise;       // Promise object
    Parameters params;          // Function parameters (copied)
    LocalVariables locals;      // Local variables
    TemporaryValues temps;      // Temporary values
};
```

Example:

```
Task<int> compute(int x) {
    int y = 10;                 // Local variable
    auto result = co_await async_add(x, y);  // Suspension point
    co_return result * 2;
}

// Compiler transforms this roughly to:
struct compute_CoroutineFrame {
    void* resume_point;
    Task<int>::promise_type promise;
    int x;                      // Parameter
    int y;                      // Local variable
    int result;                 // Temporary across suspension
};
```

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 4. Promise Types - Customizing Coroutine Behavior (1/3)

What is a Promise Type?

The **promise type** is a class that customizes how a coroutine behaves. It controls:
- What happens at coroutine start (`initial_suspend`)
- What happens at coroutine end (`final_suspend`)
- How return values are handled (`return_value` / `return_void`)
- How exceptions are handled (`unhandled_exception`)
- What the coroutine returns to the caller (`get_return_object`)

Promise Type Requirements:

```
struct promise_type {
    // Required: Create the return object
    ReturnType get_return_object();

    // Required: Initial suspension behavior
    std::suspend_always initial_suspend();  // or suspend_never

    // Required: Final suspension behavior
    std::suspend_always final_suspend() noexcept;

    // Required: Handle completion (choose one)
    void return_void();              // For coroutines that don't return values
    // OR
    void return_value(T value);      // For coroutines that return values

    // Required: Handle uncaught exceptions
    void unhandled_exception();

    // Optional: Handle co_yield
    std::suspend_always yield_value(T value);
};
```

Finding the Promise Type:

The compiler looks for `promise_type` in the return type:

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 4. Promise Types - Customizing Coroutine Behavior (2/3)

```
// If coroutine returns Task<int>:
Task<int> my_coroutine() {
    co_return 42;
}

// Compiler looks for:
typename Task<int>::promise_type

// So Task<T> must define:
template<typename T>
struct Task {
    struct promise_type {
        // ... promise interface
    };
};
```

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 4. Promise Types - Customizing Coroutine Behavior (3/3)

Example: Simple Task Promise:

```
template<typename T>
struct Task {
    struct promise_type {
        T value_;
        std::exception_ptr exception_;

        Task get_return_object() {
            return Task{std::coroutine_handle<promise_type>::from_promise(*this)};
        }

        std::suspend_always initial_suspend() { return {}; }  // Start suspended
        std::suspend_always final_suspend() noexcept { return {}; }  // Stay suspended at end

        void return_value(T val) {
            value_ = std::move(val);
        }

        void unhandled_exception() {
            exception_ = std::current_exception();
        }
    };

    std::coroutine_handle<promise_type> handle_;

    T get() {
        handle_.resume();  // Resume coroutine to completion
        if (handle_.promise().exception_)
            std::rethrow_exception(handle_.promise().exception_);
        return std::move(handle_.promise().value_);
    }
};
```

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 5. Awaitables and co_await - Suspension Points (1/3)

What is an Awaitable?

An awaitable is any type that can be used with `co_await`. It controls:
- Whether to suspend (`await_ready()`)
- What to do when suspending (`await_suspend()`)
- What value to produce when resuming (`await_resume()`)

Awaitable Interface:

```
struct MyAwaitable {
    // Called first: Should we suspend?
    bool await_ready() const noexcept {
        return false;  // true = don't suspend, false = suspend
    }

    // Called if suspending: What to do when suspended?
    void await_suspend(std::coroutine_handle<> handle) {
        // Schedule resumption (e.g., on thread pool, after I/O, etc.)
        // Option 1: Return void (always suspend)
        // Option 2: Return bool (true = suspend, false = resume immediately)
        // Option 3: Return another coroutine_handle (symmetric transfer)
    }

    // Called when resumed: What value to produce?
    T await_resume() const noexcept {
        return result_;  // This is the value of the co_await expression
    }
};

// Usage:
Task<int> example() {
    int value = co_await MyAwaitable{};  // value = await_resume()'s return
}
```

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 5. Awaitables and co_await - Suspension Points (2/3)

Standard Awaitables:

```
// std::suspend_always: Always suspends
struct suspend_always {
    bool await_ready() const noexcept { return false; }  // Always suspend
    void await_suspend(std::coroutine_handle<>) const noexcept {}
    void await_resume() const noexcept {}
};

// std::suspend_never: Never suspends
struct suspend_never {
    bool await_ready() const noexcept { return true; }  // Never suspend
    void await_suspend(std::coroutine_handle<>) const noexcept {}
    void await_resume() const noexcept {}
};
```

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 5. Awaitables and co_await - Suspension Points (3/3)

Example: Timer Awaitable:

```
struct TimerAwaitable {
    std::chrono::milliseconds duration_;

    bool await_ready() const noexcept {
        return duration_.count() == 0;  // Don't suspend if duration is 0
    }

    void await_suspend(std::coroutine_handle<> handle) {
        // Schedule resumption after duration
        std::thread([handle, dur = duration_]() {
            std::this_thread::sleep_for(dur);
            handle.resume();  // Resume the coroutine
        }).detach();
    }

    void await_resume() const noexcept {}
};

// Usage:
Task<void> delayed_task() {
    std::cout << "Starting\n";
    co_await TimerAwaitable{std::chrono::seconds(2)};  // Suspend for 2 seconds
    std::cout << "Resumed after 2 seconds\n";
}
```

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 6. Generators with co_yield - Producing Sequences (1/4)

Generator Pattern:

A generator is a coroutine that produces a sequence of values lazily using `co_yield`.

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 6. Generators with co_yield - Producing Sequences (2/4)

Simple Generator Implementation:

```
template<typename T>
struct Generator {
    struct promise_type {
        T current_value_;

        Generator get_return_object() {
            return Generator{std::coroutine_handle<promise_type>::from_promise(*this)};
        }

        std::suspend_always initial_suspend() { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }

        std::suspend_always yield_value(T value) {
            current_value_ = std::move(value);
            return {};  // Suspend after storing value
        }

        void return_void() {}  // Generators don't return values

        void unhandled_exception() {
            std::terminate();
        }
    };

    std::coroutine_handle<promise_type> handle_;

    Generator(std::coroutine_handle<promise_type> h) : handle_(h) {}
    ~Generator() { if (handle_) handle_.destroy(); }

    // Iterator interface for range-based for
    struct iterator {
        std::coroutine_handle<promise_type> handle_;

        iterator& operator++() {
            handle_.resume();
            if (handle_.done()) handle_ = nullptr;
            return *this;
        }

// ...continues on next slide
```

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 6. Generators with co_yield - Producing Sequences (3/4)

[Code continues across slides]

```
// ...continued from previous slide
        T operator*() const {
            return handle_.promise().current_value_;
        }

        bool operator==(const iterator& other) const {
            return handle_ == other.handle_;
        }
    };

    iterator begin() {
        handle_.resume();  // Start the generator
        if (handle_.done()) return {nullptr};
        return {handle_};
    }

    iterator end() {
        return {nullptr};
    }
};

// Usage:
Generator<int> fibonacci() {
    int a = 0, b = 1;
    while (true) {
        co_yield a;
        int next = a + b;
        a = b;
        b = next;
    }
}

int main() {
    int count = 0;
    for (int fib : fibonacci()) {
        std::cout << fib << " ";
        if (++count == 10) break;
    }
    // Output: 0 1 1 2 3 5 8 13 21 34
}
```

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 6. Generators with co_yield - Producing Sequences (4/4)

How co_yield Works:

```
Generator<int> numbers() {
    co_yield 1;  // Suspend here, return 1 to caller
    co_yield 2;  // Resume here, suspend again, return 2
    co_yield 3;  // Resume here, suspend again, return 3
}

// Execution flow:
auto gen = numbers();           // Creates coroutine, suspends at initial_suspend
auto it = gen.begin();          // Resumes, runs to first co_yield
std::cout << *it;               // 1 (from yield_value)
++it;                           // Resumes, runs to second co_yield
std::cout << *it;               // 2
++it;                           // Resumes, runs to third co_yield
std::cout << *it;               // 3
++it;                           // Resumes, reaches end
// it == gen.end() (done)
```

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 7. Coroutine Handles - Manual Control (1/3)

What is a Coroutine Handle?

`std::coroutine_handle<PromiseType>` is a low-level handle to a suspended coroutine that allows you to:
- Resume the coroutine (`handle.resume()`)
- Check if it's done (`handle.done()`)
- Destroy the coroutine (`handle.destroy()`)
- Access the promise (`handle.promise()`)

Coroutine Handle Interface:

```
template<typename Promise = void>
struct coroutine_handle {
    // Create from promise
    static coroutine_handle from_promise(Promise& promise);

    // Resume execution
    void resume();

    // Check if coroutine finished
    bool done() const;

    // Destroy coroutine frame
    void destroy();

    // Access promise
    Promise& promise() const;  // Only for coroutine_handle<Promise>

    // Conversion to bool
    explicit operator bool() const;
};
```

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 7. Coroutine Handles - Manual Control (2/3)

Example: Manual Coroutine Control:

```
Task<int> async_computation() {
    std::cout << "Step 1\n";
    co_await std::suspend_always{};

    std::cout << "Step 2\n";
    co_await std::suspend_always{};

    std::cout << "Step 3\n";
    co_return 42;
}

int main() {
    auto task = async_computation();
    auto handle = task.get_handle();

    std::cout << "Starting\n";
    handle.resume();  // Prints "Step 1", then suspends
    std::cout << "After first resume\n";

    handle.resume();  // Prints "Step 2", then suspends
    std::cout << "After second resume\n";

    handle.resume();  // Prints "Step 3", completes
    std::cout << "After third resume\n";

    if (handle.done()) {
        std::cout << "Coroutine finished\n";
    }

    handle.destroy();  // Clean up
}

/* Output:
Starting
Step 1
After first resume
Step 2
After second resume
Step 3
// ...continues on next slide
```

📌 7. Coroutine Handles - Manual Control (3/3)

[Code continues across slides]

```
// ...continued from previous slide
After third resume
Coroutine finished
*/
```

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 8. Symmetric Transfer - Zero-Overhead Coroutine Chains

The Problem: Stack Overflow in Coroutine Chains:

```
Task<int> recursive_task(int n) {
    if (n == 0) co_return 1;
    auto result = co_await recursive_task(n - 1);  // Potential stack overflow!
    co_return result + 1;
}

// Calling recursive_task(10000) can overflow the stack
```

The Solution: Symmetric Transfer:

By returning a `coroutine_handle` from `await_suspend`, you can **transfer control directly** to another coroutine without growing the stack:

```
struct TaskAwaitable {
    std::coroutine_handle<> next_handle_;

    bool await_ready() const noexcept { return false; }

    // Return coroutine_handle for symmetric transfer
    std::coroutine_handle<> await_suspend(std::coroutine_handle<> current) noexcept {
        return next_handle_;  // Transfer directly to next coroutine
    }

    int await_resume() const noexcept { return 0; }
};

// Now recursive_task won't overflow
```

Key Benefit: Allows arbitrary-depth coroutine chains without stack growth (tail-call optimization for coroutines).

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 9. Exception Handling in Coroutines (1/2)

Exceptions in Coroutines:

```
Task<int> may_throw() {
    if (some_condition) {
        throw std::runtime_error("Error!");
    }
    co_return 42;
}

Task<int> caller() {
    try {
        int result = co_await may_throw();
        co_return result;
    } catch (const std::exception& e) {
        std::cout << "Caught: " << e.what() << '\n';
        co_return -1;
    }
}
```

How It Works:

1. If exception is thrown in coroutine body:
2. `promise.unhandled_exception()` is called
3. Promise can store the exception: `exception_ = std::current_exception();`
4. When coroutine is resumed or result is retrieved:
5. Check if exception exists: `if (promise.exception_) std::rethrow_exception(promise.exception_);`

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 9. Exception Handling in Coroutines (2/2)

Promise Implementation:

```
struct promise_type {
    std::exception_ptr exception_;

    void unhandled_exception() {
        exception_ = std::current_exception();
    }

    T get_result() {
        if (exception_)
            std::rethrow_exception(exception_);
        return value_;
    }
};
```

C++20 Coroutines - Stackless Cooperative Multitasking

Theory

📌 10. Coroutine Memory Management (1/2)

Heap Allocation:

By default, coroutines allocate their frame on the **heap**:

```
Task<int> my_coroutine() {
    // Coroutine frame allocated on heap
    co_return 42;
}
```

Elision Optimization:

The compiler can elide heap allocation if:
- The coroutine's lifetime is nested within the caller
- The size of the frame is known at compile time

```
Task<int> inner() { co_return 42; }

void outer() {
    auto task = inner();
    auto result = task.get();  // May elide heap allocation
}
```

Custom Allocation:

You can provide custom `operator new` and `operator delete` in the promise:

```
struct promise_type {
    void* operator new(std::size_t size) {
        std::cout << "Allocating " << size << " bytes\n";
        return ::operator new(size);
    }

    void operator delete(void* ptr) {
        std::cout << "Deallocating\n";
        ::operator delete(ptr);
    }

    // ... rest of promise interface
};
```

📌 10. Coroutine Memory Management (2/2)

HALO Optimization (Heap Allocation eLision Optimization):

Modern compilers can often eliminate heap allocation entirely for short-lived coroutines.

C++20 Coroutines - Stackless Cooperative Multitasking

Edge Cases

📌 1. Dangling References in Coroutines

The Problem:

```
Task<void> process(const std::string& data) {
    co_await std::suspend_always{};
    std::cout << data << '\n';  // ❌ DANGER: `data` may be dangling!
}

void caller() {
    std::string temp = "temporary";
    auto task = process(temp);  // `temp` is captured by reference
    // temp is destroyed here
    task.resume();  // ❌ UB: `data` in coroutine refers to destroyed object
}
```

Why It Happens:

Coroutine parameters are stored in the coroutine frame. If they're references, they can dangle when the referenced object is destroyed before the coroutine completes.

Solution:

```
// ✅ Pass by value
Task<void> process(std::string data) {
    co_await std::suspend_always{};
    std::cout << data << '\n';  // Safe: `data` is copied into frame
}
```

C++20 Coroutines - Stackless Cooperative Multitasking

Edge Cases

📌 2. Forgetting to Resume/Destroy

The Problem:

```
Task<int> leak_coroutine() {
    co_return 42;
}

int main() {
    auto task = leak_coroutine();
    // ❌ Never resumed or destroyed - memory leak!
}
```

Why It Happens:

Creating a coroutine allocates a frame. If you never resume it to completion or explicitly destroy it, the frame leaks.

Solution:

```
// ✅ RAII wrapper ensures cleanup
struct Task {
    std::coroutine_handle<promise_type> handle_;

    ~Task() {
        if (handle_) handle_.destroy();  // Automatic cleanup
    }

    Task(Task&& other) : handle_(std::exchange(other.handle_, {})) {}
    Task& operator=(Task&& other) {
        if (handle_) handle_.destroy();
        handle_ = std::exchange(other.handle_, {});
        return *this;
    }

    // Delete copy
    Task(const Task&) = delete;
    Task& operator=(const Task&) = delete;
};
```

C++20 Coroutines - Stackless Cooperative Multitasking

Edge Cases

📌 3. co_await in Catch Block

The Problem:

```
Task<void> example() {
    try {
        // some code
    } catch (...) {
        co_await cleanup();  // ❌ Compiler error in C++20!
    }
}
```

Why It Fails:

C++20 doesn't allow `co_await` in catch blocks due to implementation complexity.

Solution:

```
// ✅ Workaround: Flag the error, handle outside try block
Task<void> example() {
    bool error_occurred = false;
    try {
        // some code
    } catch (...) {
        error_occurred = true;
    }

    if (error_occurred) {
        co_await cleanup();
    }
}
```

C++20 Coroutines - Stackless Cooperative Multitasking

Edge Cases

📌 4. Returning Coroutine Type with Deduced Return Type

The Problem:

```
// ❌ Compiler error: can't deduce coroutine return type
auto my_coroutine() {
    co_return 42;
}
```

Why It Fails:

The compiler needs to know the promise type before parsing the coroutine body, but `auto` requires parsing the body first (chicken-and-egg problem).

Solution:

```
// ✅ Explicit return type
Task<int> my_coroutine() {
    co_return 42;
}
```

C++20 Coroutines - Stackless Cooperative Multitasking

Edge Cases

📌 5. Coroutine Constructors/Destructors

The Problem:

```
struct MyClass {
    MyClass() {
        co_return;  // ❌ Compiler error: constructors can't be coroutines
    }

    ~MyClass() {
        co_return;  // ❌ Compiler error: destructors can't be coroutines
    }
};
```

Why It Fails:

Constructors and destructors have special semantics (object initialization/cleanup) that conflict with coroutine suspension.

Workaround:

```
struct MyClass {
    Task<void> init() {  // ✅ Separate init coroutine
        // async initialization
        co_return;
    }
};
```

📌 6. co_yield in Functions Returning Non-Generator Types

The Problem:

```
Task<int> broken() {
    co_yield 1;  // ❌ Task doesn't support co_yield!
}
```

Why It Fails:

`co_yield` requires the promise type to have a `yield_value()` method. If the return type's promise doesn't provide it, compilation fails.

Solution:

```
// ✅ Use Generator type that supports co_yield
Generator<int> working() {
    co_yield 1;
    co_yield 2;
}
```

C++20 Coroutines - Stackless Cooperative Multitasking

Edge Cases

📌 7. Mixing co_return Value and co_return Void

The Problem:

```
Task<int> inconsistent() {
    if (condition) {
        co_return 42;  // return_value(int)
    } else {
        co_return;     // return_void()
    }
}
// ❌ Promise must provide either return_value OR return_void, not both
```

Solution:

```
// ✅ Always return a value
Task<int> consistent() {
    if (condition) {
        co_return 42;
    } else {
        co_return 0;  // Explicit default value
    }
}
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 1. Async Task with Result (1/2)

```
#include <coroutine>
#include <iostream>
#include <exception>
#include <thread>

template<typename T>
struct Task {
    struct promise_type {
        T value_;
        std::exception_ptr exception_;

        Task get_return_object() {
            return Task{std::coroutine_handle<promise_type>::from_promise(*this)};
        }

        std::suspend_always initial_suspend() { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }

        void return_value(T val) {
            value_ = std::move(val);
        }

        void unhandled_exception() {
            exception_ = std::current_exception();
        }
    };

    std::coroutine_handle<promise_type> handle_;

    Task(std::coroutine_handle<promise_type> h) : handle_(h) {}
    ~Task() { if (handle_) handle_.destroy(); }

    Task(Task&& other) : handle_(std::exchange(other.handle_, {})) {}

    T get() {
        if (!handle_.done()) {
            handle_.resume();
        }

// ...continues on next slide
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 1. Async Task with Result (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        if (handle_.promise().exception_) {
            std::rethrow_exception(handle_.promise().exception_);
        }

        return std::move(handle_.promise().value_);
    }
};

// Usage
Task<int> compute_async() {
    std::cout << "Computing...\n";
    co_return 42;
}

int main() {
    auto task = compute_async();
    int result = task.get();
    std::cout << "Result: " << result << '\n';
}
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 2. Generator for Infinite Sequences (1/3)

```
#include <coroutine>
#include <iostream>

template<typename T>
struct Generator {
    struct promise_type {
        T current_value_;
        std::exception_ptr exception_;

        Generator get_return_object() {
            return Generator{std::coroutine_handle<promise_type>::from_promise(*this)};
        }

        std::suspend_always initial_suspend() { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }

        std::suspend_always yield_value(T value) {
            current_value_ = std::move(value);
            return {};
        }

        void return_void() {}

        void unhandled_exception() {
            exception_ = std::current_exception();
        }
    };

    std::coroutine_handle<promise_type> handle_;

    Generator(std::coroutine_handle<promise_type> h) : handle_(h) {}
    ~Generator() { if (handle_) handle_.destroy(); }

    Generator(Generator&& other) : handle_(std::exchange(other.handle_, {})) {}

    struct iterator {
        std::coroutine_handle<promise_type> handle_;

        iterator& operator++() {
// ...continues on next slide
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 2. Generator for Infinite Sequences (2/3)

```
// ...continued from previous slide
            handle_.resume();
            if (handle_.done()) handle_ = nullptr;
            return *this;
        }

        T operator*() const {
            return handle_.promise().current_value_;
        }

        bool operator!=(const iterator& other) const {
            return handle_ != other.handle_;
        }
    };

    iterator begin() {
        handle_.resume();
        if (handle_.done()) return {nullptr};
        return {handle_};
    }

    iterator end() {
        return {nullptr};
    }
};

// Generate Fibonacci sequence
Generator<int> fibonacci() {
    int a = 0, b = 1;
    while (true) {
        co_yield a;
        int next = a + b;
        a = b;
        b = next;
    }
}

int main() {
    int count = 0;
    for (int fib : fibonacci()) {
        std::cout << fib << " ";
// ...continues on next slide
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 2. Generator for Infinite Sequences (3/3)

[Code continues across slides]

```
// ...continued from previous slide
        if (++count == 15) break;
    }
    std::cout << '\n';
}

/* Output:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
*/
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 3. Async Timer (1/2)

```
#include <coroutine>
#include <iostream>
#include <thread>
#include <chrono>

template<typename T>
struct Task {
    // ... same as Example 1
};

struct TimerAwaitable {
    std::chrono::milliseconds duration_;

    bool await_ready() const noexcept {
        return duration_.count() == 0;
    }

    void await_suspend(std::coroutine_handle<> handle) {
        std::thread([handle, dur = duration_]() {
            std::this_thread::sleep_for(dur);
            handle.resume();
        }).detach();
    }

    void await_resume() const noexcept {}
};

Task<void> delayed_message() {
    using namespace std::chrono_literals;

    std::cout << "Starting...\n";
    co_await TimerAwaitable{1000ms};
    std::cout << "After 1 second\n";
    co_await TimerAwaitable{1000ms};
    std::cout << "After 2 seconds\n";
    co_await TimerAwaitable{1000ms};
    std::cout << "After 3 seconds\n";
}

// ...continues on next slide
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 3. Async Timer (2/2)

[Code continues across slides]

```
// ...continued from previous slide
int main() {
    auto task = delayed_message();
    task.get();
}

/* Output:
Starting...
(1 second pause)
After 1 second
(1 second pause)
After 2 seconds
(1 second pause)
After 3 seconds
*/
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 4. Lazy Sequence Processing (1/2)

```
#include <coroutine>
#include <iostream>
#include <vector>

template<typename T>
struct Generator {
    // ... same as Example 2
};

// Lazy filter
Generator<int> filter(Generator<int> gen, auto pred) {
    for (int val : gen) {
        if (pred(val)) {
            co_yield val;
        }
    }
}

// Lazy map
Generator<int> map(Generator<int> gen, auto func) {
    for (int val : gen) {
        co_yield func(val);
    }
}

// Lazy take
Generator<int> take(Generator<int> gen, int n) {
    int count = 0;
    for (int val : gen) {
        if (count++ >= n) break;
        co_yield val;
    }
}

// Generate natural numbers
Generator<int> naturals() {
    int n = 1;
    while (true) {
        co_yield n++;
// ...continues on next slide
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 4. Lazy Sequence Processing (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    }
}

int main() {
    // Pipeline: naturals → filter evens → map (square) → take 5
    auto pipeline = take(
        map(
            filter(naturals(), [](int n) { return n % 2 == 0; }),
            [](int n) { return n * n; }
        ),
        5
    );

    for (int val : pipeline) {
        std::cout << val << " ";
    }
    std::cout << '\n';
}

/* Output:
4 16 36 64 100
*/
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 5. Coroutine-Based State Machine (1/3)

```
#include <coroutine>
#include <iostream>
#include <string>

enum class State { START, PROCESSING, WAITING, DONE };

struct StateMachine {
    struct promise_type {
        State current_state_ = State::START;

        StateMachine get_return_object() {
            return StateMachine{std::coroutine_handle<promise_type>::from_promise(*this)};
        }

        std::suspend_always initial_suspend() { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }

        std::suspend_always yield_value(State state) {
            current_state_ = state;
            return {};
        }

        void return_void() {}
        void unhandled_exception() {}
    };

    std::coroutine_handle<promise_type> handle_;

    StateMachine(std::coroutine_handle<promise_type> h) : handle_(h) {}
    ~StateMachine() { if (handle_) handle_.destroy(); }

    State step() {
        if (!handle_.done()) {
            handle_.resume();
        }
        return handle_.promise().current_state_;
    }

    bool done() const {
// ...continues on next slide
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 5. Coroutine-Based State Machine (2/3)

```
// ...continued from previous slide
        return handle_.done();
    }
};

StateMachine workflow() {
    std::cout << "Starting workflow\n";
    co_yield State::START;

    std::cout << "Processing data\n";
    co_yield State::PROCESSING;

    std::cout << "Waiting for input\n";
    co_yield State::WAITING;

    std::cout << "Workflow complete\n";
    co_yield State::DONE;
}

int main() {
    auto machine = workflow();

    while (!machine.done()) {
        State state = machine.step();

        switch (state) {
            case State::START:
                std::cout << "State: START\n";
                break;
            case State::PROCESSING:
                std::cout << "State: PROCESSING\n";
                break;
            case State::WAITING:
                std::cout << "State: WAITING\n";
                break;
            case State::DONE:
                std::cout << "State: DONE\n";
                break;
        }
    }
}
// ...continues on next slide
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 5. Coroutine-Based State Machine (3/3)

[Code continues across slides]

```
// ...continued from previous slide

/* Output:
Starting workflow
State: START
Processing data
State: PROCESSING
Waiting for input
State: WAITING
Workflow complete
State: DONE
*/
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 6. Recursive Coroutine (with Symmetric Transfer) (1/2)

```
#include <coroutine>
#include <iostream>

template<typename T>
struct Task {
    struct promise_type {
        T value_;

        Task get_return_object() {
            return Task{std::coroutine_handle<promise_type>::from_promise(*this)};
        }

        std::suspend_always initial_suspend() { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }

        void return_value(T val) {
            value_ = std::move(val);
        }

        void unhandled_exception() {}
    };

    std::coroutine_handle<promise_type> handle_;

    struct awaitable {
        std::coroutine_handle<promise_type> next_;

        bool await_ready() const noexcept { return false; }

        std::coroutine_handle<> await_suspend(std::coroutine_handle<> current) noexcept {
            return next_;  // Symmetric transfer
        }

        T await_resume() const noexcept {
            return next_.promise().value_;
        }
    };

    awaitable operator co_await() {
// ...continues on next slide
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 6. Recursive Coroutine (with Symmetric Transfer) (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        return awaitable{handle_};
    }

    T get() {
        handle_.resume();
        return handle_.promise().value_;
    }
};

Task<int> factorial(int n) {
    if (n <= 1) {
        co_return 1;
    }

    int prev = co_await factorial(n - 1);  // Symmetric transfer prevents stack overflow
    co_return n * prev;
}

int main() {
    auto task = factorial(10);
    std::cout << "10! = " << task.get() << '\n';
}

/* Output:
10! = 3628800
*/
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 7. Cooperative Multitasking Scheduler (1/3)

```
#include <coroutine>
#include <iostream>
#include <queue>
#include <memory>

class Scheduler {
    std::queue<std::coroutine_handle<>> ready_queue_;

public:
    void schedule(std::coroutine_handle<> handle) {
        ready_queue_.push(handle);
    }

    void run() {
        while (!ready_queue_.empty()) {
            auto handle = ready_queue_.front();
            ready_queue_.pop();

            handle.resume();

            if (!handle.done()) {
                ready_queue_.push(handle);  // Re-queue if not done
            }
        }
    }
};

inline Scheduler scheduler;

struct YieldAwaitable {
    bool await_ready() const noexcept { return false; }

    void await_suspend(std::coroutine_handle<> handle) {
        scheduler.schedule(handle);  // Re-schedule this coroutine
    }

    void await_resume() const noexcept {}
};

// ...continues on next slide
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 7. Cooperative Multitasking Scheduler (2/3)

```
// ...continued from previous slide
template<typename T>
struct Task {
    struct promise_type {
        T value_;

        Task get_return_object() {
            return Task{std::coroutine_handle<promise_type>::from_promise(*this)};
        }

        std::suspend_never initial_suspend() { return {}; }  // Start immediately
        std::suspend_always final_suspend() noexcept { return {}; }

        void return_value(T val) {
            value_ = std::move(val);
        }

        void unhandled_exception() {}
    };

    std::coroutine_handle<promise_type> handle_;
};

Task<int> worker(int id) {
    for (int i = 0; i < 3; ++i) {
        std::cout << "Worker " << id << " - iteration " << i << '\n';
        co_await YieldAwaitable{};  // Yield to other tasks
    }
    co_return id;
}

int main() {
    auto task1 = worker(1);
    auto task2 = worker(2);
    auto task3 = worker(3);

    scheduler.run();
}

/* Output:
Worker 1 - iteration 0
// ...continues on next slide
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 7. Cooperative Multitasking Scheduler (3/3)

[Code continues across slides]

```
// ...continued from previous slide
Worker 2 - iteration 0
Worker 3 - iteration 0
Worker 1 - iteration 1
Worker 2 - iteration 1
Worker 3 - iteration 1
Worker 1 - iteration 2
Worker 2 - iteration 2
Worker 3 - iteration 2
*/
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 8. Async File Reader (1/2)

```
#include <coroutine>
#include <iostream>
#include <fstream>
#include <string>
#include <thread>

template<typename T>
struct Task {
    // ... (same as previous examples)
};

struct FileReadAwaitable {
    std::string filename_;
    std::string result_;

    bool await_ready() const noexcept { return false; }

    void await_suspend(std::coroutine_handle<> handle) {
        std::thread([this, handle]() {
            std::ifstream file(filename_);
            if (file.is_open()) {
                std::string line;
                while (std::getline(file, line)) {
                    result_ += line + "\n";
                }
            }
            handle.resume();
        }).detach();
    }

    std::string await_resume() const noexcept {
        return result_;
    }
};

Task<void> process_file(const std::string& filename) {
    std::cout << "Reading file: " << filename << '\n';

    auto content = co_await FileReadAwaitable{filename};
// ...continues on next slide
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 8. Async File Reader (2/2)

[Code continues across slides]

```
// ...continued from previous slide

    std::cout << "File content (" << content.size() << " bytes):\n";
    std::cout << content << '\n';
}

int main() {
    auto task = process_file("example.txt");
    task.get();
}
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 9. Generator with Transformation (1/2)

```
#include <coroutine>
#include <iostream>
#include <vector>

template<typename T>
struct Generator {
    // ... (same as Example 2)
};

// Generate squares
Generator<int> squares(int max) {
    for (int i = 1; i <= max; ++i) {
        co_yield i * i;
    }
}

// Generate cubes
Generator<int> cubes(int max) {
    for (int i = 1; i <= max; ++i) {
        co_yield i * i * i;
    }
}

// Interleave two generators
Generator<int> interleave(Generator<int> g1, Generator<int> g2) {
    auto it1 = g1.begin();
    auto it2 = g2.begin();
    auto end1 = g1.end();
    auto end2 = g2.end();

    while (it1 != end1 || it2 != end2) {
        if (it1 != end1) {
            co_yield *it1;
            ++it1;
        }
        if (it2 != end2) {
            co_yield *it2;
            ++it2;
        }
// ...continues on next slide
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 9. Generator with Transformation (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    }
}

int main() {
    auto combined = interleave(squares(5), cubes(5));

    for (int val : combined) {
        std::cout << val << " ";
    }
    std::cout << '\n';
}

/* Output:
1 1 4 8 9 27 16 64 25 125
(squares: 1, 4, 9, 16, 25)
(cubes: 1, 8, 27, 64, 125)
*/
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 10. Coroutine-Based Event Loop (1/3)

```
#include <coroutine>
#include <iostream>
#include <queue>
#include <chrono>
#include <thread>

class EventLoop {
    struct TimedTask {
        std::coroutine_handle<> handle;
        std::chrono::steady_clock::time_point wake_time;

        bool operator<(const TimedTask& other) const {
            return wake_time > other.wake_time;  // Min-heap
        }
    };

    std::priority_queue<TimedTask> timers_;

public:
    void schedule_after(std::coroutine_handle<> handle, std::chrono::milliseconds delay) {
        auto wake_time = std::chrono::steady_clock::now() + delay;
        timers_.push({handle, wake_time});
    }

    void run() {
        while (!timers_.empty()) {
            auto task = timers_.top();
            timers_.pop();

            auto now = std::chrono::steady_clock::now();
            if (task.wake_time > now) {
                std::this_thread::sleep_for(task.wake_time - now);
            }

            task.handle.resume();

            if (!task.handle.done()) {
                // Could re-schedule if needed
            }
// ...continues on next slide
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 10. Coroutine-Based Event Loop (2/3)

```
// ...continued from previous slide
        }
    }
};

inline EventLoop event_loop;

struct SleepAwaitable {
    std::chrono::milliseconds duration_;

    bool await_ready() const noexcept { return false; }

    void await_suspend(std::coroutine_handle<> handle) {
        event_loop.schedule_after(handle, duration_);
    }

    void await_resume() const noexcept {}
};

template<typename T>
struct Task {
    // ... (same promise_type as before)
};

Task<void> task1() {
    using namespace std::chrono_literals;

    std::cout << "Task 1: Start\n";
    co_await SleepAwaitable{500ms};
    std::cout << "Task 1: After 500ms\n";
    co_await SleepAwaitable{500ms};
    std::cout << "Task 1: Done\n";
}

Task<void> task2() {
    using namespace std::chrono_literals;

    std::cout << "Task 2: Start\n";
    co_await SleepAwaitable{300ms};
    std::cout << "Task 2: After 300ms\n";
    co_await SleepAwaitable{700ms};
// ...continues on next slide
```

C++20 Coroutines - Stackless Cooperative Multitasking

Code Examples

📌 10. Coroutine-Based Event Loop (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    std::cout << "Task 2: Done\n";
}

int main() {
    auto t1 = task1();
    auto t2 = task2();

    event_loop.run();
}

/* Output (with timestamps):
Task 1: Start
Task 2: Start
(300ms)
Task 2: After 300ms
(200ms)
Task 1: After 500ms
(500ms)
Task 1: Done
(200ms)
Task 2: Done
*/
```

What You Learned

Key Takeaways

- ✓ The Problem They Solve
- ✓ The Three Keywords
- ✓ How They Work Under the Hood
- ✓ Customizing Coroutine Behavior

→ Swipe to learn →